ShanghaiTech University

CS101 Algorithms and Data Structures
Fall 2024

Homework 3

Wenye Xiong 2023533141

Due date: October 23, 2024, at 23:59

1. Please write your solutions in English.
2. Submit your solutions to Gradescope.
3. Set your FULL name to your Chinese name and your STUDENT ID correctly in Gradescope account settings.
4. If you want to submit a handwritten version, scan it clearly. `CamScanner` is recommended.
5. We recommend you to write in LaTeX.
6. When submitting, match your solutions to the problems correctly.
7. No late submission will be accepted.
8. Violations to any of the above may result in zero points.

Important Notes:

1. Some problems in this homework requires you to design divide-and-conquer algorithm. When grading these problems, we will put more emphasis on how you reduce a problem to a smaller size problem and how to combine their solutions with divide-and-conquer strategy.

2. There are mainly 3 kinds of writing problems that may be involved in this assignment (and following assignments). We recommend you to know what each part **should** include:

    1. Describe/Design an algorithm that...: Clear description of your algorithm design in natural language is needed. If you want to present it into pseudocode, you'd better take it seriously on your boundary cases and supplement it with meaningful, clear variable names. Unless the problem allows, you should give a complete algorithm instead of fragmented words incremented by some other algorithm.
    2. Justify your algorithm/Justify its correctness/...: You should give out a proof of why your algorithm is correct. The requirements may not be so strict, but please at least clarify every step of your proof and make sure they are reasonable. You can give out an answer or some propositions and then make **reliable and credible** explanations, and we encourage you to do that on suitable cases (e.g. using induction).
    3. Analyze the time complexity/Show its time complexity is ...: You should give out a exact time complexity and some reason to justify it (e.g. Master Theorem/Recurrence relation/...). We won't restrict too much on O and Θ. However, your time complexity bound should be **tight**. For example, we won't make it correct when you say something is $O(n^2)$ with its worst case running on $\Theta(n \log n)$ (Even if it is logically correct).

3. Your answer for these problems is not allowed to include real C or C++ code.

4. In your description of algorithm design, you should describe each step of your algorithm clearly.

5. You are encouraged to write pseudocode to facilitate explaining your algorithm design, though this is not mandatory. However, we recommend you not to write pseudocode, if you are using handwriting to finish your homework. If you choose to write pseudocode, please give some additional descriptions to make your pseudocode intelligible.

6. You are recommended to finish the algorithm design part of this homework with LaTeX. If you insist on using handwriting to finish it, your explanation should be at least tidy and readable (can be specified) otherwise your assignments will be considered as unspecified.

7. Please **do not** just hand in a picture of answer-sheet with several answers only. For both of your and our convenience, please at least upload a PDF assignment file (In fact, you don't need to cut down any pages because we will filter it for you).

1. (0 points) Binary Search Example

Design an algorithm to find the index of an element x in a sorted array a of n elements.

Solution:

We will show how you are supposed to finish the 3 kinds of problems mentioned last page.

Algorithm Design: We basically ignore half of the elements just after one comparison.

1. Compare x with the middle element.
2. If x matches with the middle element, return the middle index.
3. Otherwise x is greater than the mid element, then x can only lie in right half subarray after the mid element. So we recur for right half.
4. Otherwise (x is smaller) recur for the left half.

Pseudocode(Optional):

```
1: function BINARYSEARCH(a, value, left, right)
2:     if right < left then
3:         return not found
4:     end if
5:     mid ← ⌊(right − left)/2⌋ + left
6:     if a[mid] = value then
7:         return mid
8:     end if
9:     if value < a[mid] then
10:        return binarySearch(a, value, left, mid-1)
11:    else
12:        return binarySearch(a, value, mid+1, right)
13:    end if
14: end function
```

Proof of Correctness: If x happens to be the middle element, we will find it in the first step. Otherwise, if x is greater than the middle element, then all the element in the left half subarray is less than x since the original array has already been sorted, so we just need to look for x in the right half subarray. Similarly, if x is less than the middle element, then all the element in the right subarray is greater than x, so we just need to look for x in the front list. If we still can't find x in a recursive call where $left = right$, which indicates that x is not in a, we will return $not\ found$ in the next recursive call.

Time Complexity Analysis: During each recursion, the calculation of *mid* and comparison can be done in constant time, which is $O(1)$. We ignore half of the elements after each comparison, thus we need $O(\log n)$ recursions.

$$T(n) = T(n/2) + O(1)$$

Therefore, by the Master Theorem $\log_b a = 0 = d$, so $T(n) = O(\log n)$.

2. (15 points) Multiple Choices

Each question has **one or more** correct answer(s). Select all the correct answer(s). For each question, you will get 0 points if you select one or more wrong answers, but you will get 1 point if you select a non-empty subset of the correct answers.

Write your answers in the following table.

(a)	(b)	(c)	(d)	(e)
AB	AC	AD	BCD	ABC

(a) (3') Which of the following sorting algorithm(s) is(are) stable?

A. Insertion-Sort

B. Merge-Sort

C. Quick-Sort (picking a random element as pivot)

D. None of the above

(b) (3') Which of the following statements is **NOT** true?

A. The worst case time complexity of merge-sort is $O(n^2)$.

B. Given 2 sorted lists of size m and n respectively, and we want to merge them to one sorted list by merge-sort. Then in the worst case, we need $m + n - 1$ comparisons.

C. The time complexity of quick-sort, compared with merge-sort, is less affected by the initial order of the input array.

D. Traditional implementations of merge-sort need $\Theta(n \log n)$ time when the input sequence is sorted or totally reversely sorted, but it is possible to make it $\Theta(n)$ on such input while still $\Theta(n \log n)$ on the average case.

(c) (3') You have to sort n data with very limited extra memory ($\Theta(1)$ extra memory every case). Choose sort algorithm(s) which is(are) **not** appropriate.
Hint: Note that too much recursion will take use of huge stack memory.

A. Merge-sort.

B. Insertion-sort.

C. Bubble-sort.

D. Quick-sort.

(d) (3') Which of the following implementations of quick-sort may improve the average **behavior** (not only improvement on time complexity) of trivial quick-sort?(Compared to always picking the first element as pivot and using no optimization, data are uniformly distributed.)

A. Always picking the last element.

B. When partitioning the subarray $\langle a_l, \cdots, a_r \rangle$ (assuming $r - l \geqslant 2$), choose the median of $\{a_x, a_y, a_z\}$ as the pivot, where x, y, z are three different indices chosen randomly from $\{l, l+1, \cdots, r\}$.

C. When partitioning the subarray $\langle a_l, \cdots, a_r \rangle$ (assuming $r - l \geqslant 2$), we first calculate $q = \frac{1}{2}(a_{\max} + a_{\min})$ where $a_{\max}$ and $a_{\min}$ are the maximum and minimum values in the current subarray respectively. Then we traverse the whole subarray to find a_m $s.t.$ $|a_m - q| = \min_{i=l}^{r} |a_i - q|$ and choose a_m as the pivot.

D. When the disordered sub-array is short enough, use insertion-sort instead of quick-sort.

(e) (3') Which of the following statements is true?

A. If $T(n) = 2T(\frac{n}{2}) + O(\sqrt{n})$ with $T(0) = 0$ and $T(1) = 1$, then $T(n) = \Theta(n)$.

B. If $T(n) = T(\frac{n}{10}) + T(\frac{9n}{10}) + \Theta(n)$ with $T(0) = 0$ and $T(1) = 1$, then $T(n) = O(n \log n)$.

C. If $T(n) = 4T(\frac{n}{2}) + O(n^2)$ with $T(0) = 0$ and $T(1) = 1$, then $T(n) = \Theta(n^2 \log n)$.

D. If the run-time $T(n)$ of a divide-and-conquer algorithm satisfies $T(n) = aT(\frac{n}{b}) + f(n)$ with $T(0) = 0$ and $T(1) = 1$, we may deduce that the run-time for dividing the original problem into a subproblems of size $\frac{n}{b}$ is $f(n)$.

3. (15 points) Counting Jewelry Pairs

Astra opens up a jewelry store to sold the jewelry. Each jewelry has a distinct positive "beauty value" a_i, where i means it's the i-th jewelry Astra got. You are asked to help Astra to solve the problems for her customs.

(a) Breach wants to buy a pair of jewelry, where jewelry found earlier should be more beautiful i.e. have a bigger beauty value. In other words, the pair contains 2 jewelry i and j, where $i < j, a_j < a_i$.

Astra needs to count the number of jewelry pairs satisfying Breach's requirement.

i. (1') Recall what is told in class. This problem is equivalent to

◯ Quick Sort ◯ Merge Sort √ **Counting Inversions** ◯ Binary Search

ii. (2') Briefly write how to solve this problem using divide and conquer.

Solution: We divide the array into two halves and count the number of inversions in each half (let be $inv1$ and $inv2$). Then we count the number of inversions not counted in $inv1 + inv2$. This can be done during the merge step of the merge sort algorithm. For the two halves A and B, we want to find all inversions (a, b) with $a \in A$ and $b \in B$. We first sort the two halves separately. During the merge process, let i be the index of the current element in A and j be the index of the current element in B. If $A[i] > B[j]$, then there are $mid - i + 1$ inversions, where mid is the index of the last element in A. This is because all elements from $A[i]$ to $A[mid]$ are greater than $B[j]$. We increment the index of B and continue the process.

In the two questions below, you are allowed to describe your algorithm by modifying parts of the algorithm in $(a).ii$.

(b) Chamber wants to buy a pair of jewelry too. His requirement is precise: He wants that the earlier found jewelry is "quadratic more beautiful" than the other one. In other words, for two jewelry i and $j(i < j)$, Chamber wants $a_i > \alpha a_j^2 + \beta a_j + \gamma$, where a_i, a_j is the "beauty value" of i/j-th jewelry and $\alpha, \beta > 0$.
Astra needs to count the number of jewelry pairs satisfying Chambers's requirement.
You should :

i. (4') Describe your algorithm, either by modifying or rewriting one.

Solution: We can modify the counting inversions algorithm to solve this problem. We can construct another array B where $B[i] = \alpha a_i^2 + \beta a_i + \gamma$. Then during the merge step, if $A[i] > B[j]$, then there are $mid - i + 1$ inversions, where mid is the index of the last element in A. This is because all elements from $A[i]$ to $A[mid]$ are greater than $B[j]$. We increment the index of B and continue the process.

ii. (2') Justify why your algorithm is correct and analyze the time complexity.

Solution:
Proof of Correctness:
The algorithm is correct because we are counting the number of inversions where $A[i] > B[j]$, and all $B[j]$ are constructed based on the quadratic equation. Because $\alpha, \beta > 0$, the quadratic equation is always increasing as a_i increases. Therefore, if an array A is sorted in ascending order, the array B is also sorted in ascending order. So, when $A[i] > B[j]$, then all elements from $A[i]$ to $A[mid]$ are greater than $B[j]$. Therefore, we can say that the algorithm is correct.
Time Complexity Analysis: The time complexity of the algorithm is $O(n \log n)$, where n is the number of elements in the array. During each recursion, the calculation of comparison between $A[i]$ and $B[j]$ takes $O(n)$ time. So the merge step takes $O(n)$ time. We ignore half of the array in each recursion, so

$$\mathrm{T(n)} = 2\mathrm{T(n/2)} + \mathrm{O(n)}$$

Therefore, according to the Master Theorem, the time complexity of the algorithm is $O(n \log n)$.

(c) Cypher also wants to buy a pair of jewelry, too. Cypher wants to find the relation of "beauty value" between different jewelry. He wants that the difference of when the jewelry was found and the difference of beauty value are closed. In other words, for two jewelry i and $j(i < j)$, Cypher wants $a_i - a_j > \alpha|i - j|$, where a_i, a_j is the "beauty value" of i/j-th jewelry and $\alpha > 0$.

Astra needs to count the number of jewelry pairs satisfying Cypher's requirement.

You should :

i. (4') Describe your algorithm, either by modifying or rewriting one.

Solution:
We can modify the counting inversions algorithm to solve this problem. We can construct another array B where $B[i] = a_i + \alpha i$. Then we just apply our merge sort algorithm to the array B. The whole process is the same as the counting inversions algorithm, but this time the object array is B.

ii. (2') Justify why your algorithm is correct and analyze the time complexity.

Solution:
Proof of Correctness:
The algorithm is correct because we want to count the number of pairs (i, j) where $i < j, a_i - a_j > \alpha|i - j|$. Simplifying the inequality, we get $a_i - a_j > \alpha j - \alpha i$. We can rewrite this as $a_i + \alpha i > a_j + \alpha j$. Therefore, we can construct an array B where $B[i] = a_i + \alpha i$. And we can find that the number of inversions in the array B is the number of pairs (i, j) where $i < j, a_i - a_j > \alpha|i - j|$. Therefore, the algorithm is correct.
Time Complexity Analysis: The time complexity of the algorithm is $O(n \log n)$, where n is the number of elements in the array. We first do an initial calculation of the array B which takes $O(n)$ time. Then we apply the enhance merge sort algorithm to the array B. The enhance merge sort algorithm takes $O(n \log n)$ time. Therefore, the time complexity of the algorithm is $O(n \log n)$.

4. (10 points) Similarity Test

Maddelena has drawn a lot of paintings. She wants to organize her work. To finish her job better, she decides to determine whether a subset of similar paintings exists. Every paint p has its own feature λ_p.

In this problem, we specify some notations: $\mathcal{P}$ is the set of paintings. Without giving rise to ambiguity, λ_p can be referred to the feature of a painting p. $\mathcal{F} = \{\lambda_p | p \in \mathcal{P}\}$ denotes the set of features. n can be refereed to the number of paintings i.e. $n = |\mathcal{P}|$.

Note that $\mathcal{F}$ is not partially ordered i.e. you can't make comparisons for any $p, q \in \mathcal{P}$ so that you cannot sort $\mathcal{F}$ or give out inferences like $\lambda_p < \lambda_q$ or $\lambda_q < \lambda_p$. There exists an equivalence relation on $\mathcal{F}$, as $\lambda_p = \lambda_q$ indicates p and q has similar features.

The master feature is defined as the feature it is similar to **over** half of the paintings. In other words, the "master" feature is the feature$\lambda_0 \in \mathcal{F}$ s.t. $|\{p \in \mathcal{P} | \lambda_p = \lambda_0\}| > \frac{n}{2}$.

For example, 2 sets of paintings are shown: The first set of paintings has a "master" feature $\lambda_0 = \alpha$ since there exists over a half (4: a,b,e,g) paintings share this feature. The second set of paintings does not have a "master" feature because there exists no such feature that over half paintings share that.

Painting	a	b	c	d	e	f	g
Feature	α	α	β	ω	α	β	α

Table 1: A "master" feature $\lambda_0 = \alpha$.

Painting	A	B	C	D	E
Feature	α	β	ω	β	α

Table 2: No "master" feature.

(a) Maddelena wants to find a "master" feature first. She asks you to help her with those problems:

i. (3') Design a divide-and-conquer algorithm whose worst case takes $\Theta(n \log n)$ time.
Note: Of course, you can index them, but we recommend you not to.
Hint: Recall merge sort, think out the relation between the "master" features of the 2 subsets after dividing and the "master" feature of the original set. (If they have)

Solution:
Algorithm Design:
Using the divide-and-conquer algorithm, we can find the master feature of the set of paintings. The initial value of l is 0, r is n-1.
1. Divide the set of paintings into two equal halves by calculating the mid value $mid = \frac{l+r}{2}$.
2. Recursively find all the master features of the two halves.
3. If the master features of the first half is the same as the master features of the second half, then return the master feature. Otherwise, we need to determine which one(s) is the master feature.
4. To do this, we count the frequency of the master features of the first half and the second half in the input set. If count of any master feature is greater than $\frac{r-l+1}{2}$, then that master feature is the master feature of the input set.
5. If none of the master features is greater than $\frac{r-l+1}{2}$, then there is no master feature in the input set.

ii. (2') Justify it by proving its correctness and show its time complexity is $\Theta(n \log n)$.

Solution:
Proof of Correctness:
If the master feature of the first half is the same as the master feature of the second half, then because in both halves, this feature appears more than half of the time, it will also appear more than half of the time in the original set. Therefore, it is the master feature of the original set.
If the master features of the first half and the second half are different, then we need to count the frequency of the master features of the first half and the second half in the input set. If the count of any master feature is greater than $\frac{r-l+1}{2}$, then that master feature is the master feature of the input set. If none of the master features is greater than $\frac{r-l+1}{2}$, then there is no master feature in the input set.
Time Complexity Analysis:
In each recursion, we divide the set of paintings into two equal halves. The time complexity of the divide step is $O(1)$. Then in the worst case, which is everytime the master features of the two halves are different, we need to count the frequency

of the master features of the first half and the second half in the input set. The time complexity of this step is $O(n)$. Therefore, we have the following recurrence relation:

$$T(n) = 2T(n/2) + \Theta(n)$$

According to the Master Theorem, the time complexity of the algorithm in the worst case is $\Theta(n \log n)$.

(b) Maddelena finds $\Theta(n \log n)$ still not fast enough since she has so many paintings. She wants to develop a new algorithm that takes $o(n \log n)$ time.

In this problem, you can always assume that the total remaining number is even. In other words, you don't need to consider how to deal with the one superfluous after dividing.

Hint: Recall binary search, what if we give up at least half every recurrence? Match those by pairs and do differently based on whether they are similar.

i. (2') Design a divide-and-conquer algorithm that takes $o(n \log n)$ time.

Solution:

Algorithm Design:

1. Match the paintings by pairs. If two paintings are similar, then keep just one of them, otherwise, discard both.
2. Recursively apply the algorithm to the remaining paintings, till the size of the set of paintings is 1 or 0. If it is 1, then the remaining painting has the master feature. If the size is 0, then there is no master feature.

ii. (2') Justify its correctness.

Solution:

Proof of Correctness:

During each recursion, half of the paintings are discarded. If two paintings are similar, then we keep just one of them, otherwise, discard both. Therefore, the master feature is still the master feature for the remaining set. So only the remaining feature can be the master feature.

iii. (1') Analyze the time complexity of the algorithm.

Solution:

Time Complexity Analysis:

During each recursion, the time complexity of the operation is $O(n)$. The number

of paintings is halved in each recursion. Therefore, the time complexity can be expressed as:

$$T(n) = T(n/2) + O(n)$$

Therefore, by the Master Theorem, $log_b a = 0 < 1$, the time complexity of the algorithm is $O(n) = o(n \log n)$.

5. (16 points) Calculating with dividing

In this problem, we will give out some recurrence relation of the run-time $T_i(n)$. We want you to find the asymptotic order of $T_i(n)$ i.e. find a function $f(n)$ s.t. $T_i(n) = \Theta(f(n))$, depending on the recurrence relation given. You should justify your answer correctly.

As a reminder, to receive the points, please keep in mind that:

1. Use $T_i(n)$ to represent the run-time in i-th problem.
2. Make sure your upper bound for $T_i(n)$ is tight enough.
3. Make sure you have a reasonable explanation other than just giving out an assumption and verifying it by your sense.
4. For your convenience, you can always assume $T_i(n)$ is increasing. (May Not Strictly)

Hint: Guessing the upper bound of $T_i(n)$ and then proof it rigorously (e.g. using mathematical induction) is acceptable. However, simply plugging your guessed upper bound into the recurrence relation and verifying whether your guessed upper bound makes sense or not will make you lose points for reasoning. If you find it hard to prove something straight, try induction instead.

In each subproblem, you may ignore any issue arising from whether a number is an integer as well as assuming $T_i(0) = 0$ and $T_i(1) = 1$. You can make use of the Master Theorem, Recursion Tree, or other reasonable approaches to solve the following recurrence relations.

(a) (4') $T_1(n) = \begin{cases} \Theta(1), & 1 \le n \le k \\ T_1(k) + T_1(n-k) + \Theta(n), & n > k \end{cases}$. ($k$ is a constant.)

Solution: When $n > k$, we have $T_1(n) = T_1(k) + T_1(n-k) + \Theta(n) = \Theta(1) + T_1(n-k) + \Theta(n) = \Theta(n) + T_1(n-k) = \sum_{i=0}^{n/k} \Theta(n-ik) = \Theta(n^2)$.
And when $n < k$, this is just obviously right.

(b) (4') $T_2(n) = T_2(\alpha n) + T_2((1-\alpha)n) + \Theta(n)$

Solution:
Assume $T_2(n) = \Theta(n \log n)$. We can then use mathematical induction to prove it
(1) $T_2(n) = O(n \log n) \le cn \log n$
Base case: $T_2(2) = d_1 \le 2c \log 2$
We suppose for all $a < n$ $T_2(a) \le ca \log a$.
Inductive step: $T_2(n) = T_2(\alpha n) + T_2((1-\alpha)n) + \Theta(n) \le c\alpha n \log(\alpha n) + c(1-\alpha)n \log((1-\alpha)n) + \Theta(n) = c\alpha n(\log \alpha + \log n) + c(1-\alpha)n(\log(1-\alpha) + \log n) + dn = cn \log n + (c\alpha \log \alpha + c(1-\alpha)\log(1-\alpha))n + dn \le cn \log n = O(n \log n)$
(2) $T_2(n) = \Omega(n \log n)$, which can be proved in the same way.
Therefore, $T_2(n) = \Theta(n \log n)$.

(c) (4') $T_3(n) = 2T_3(\sqrt{n}) + \Theta(\log n)$.

Solution:
Assume $T_3(n) = \Theta(\log n \log\log n)$. We can then use mathematical induction to prove it
(1) $T_3(n) = O(\log n \log\log n) \le c \log n \log\log n$

Base case: $T_3(4) = d_1 \le c \log 4 \log\log 4$
Suppose for all $a < n$ $T_3(a) \le c \log a \log\log a$
Inductive step: $T_3(n) = 2T_3(\sqrt{n}) + \Theta(\log n) \le 2c \log\sqrt{n} \log\log\sqrt{n} + \Theta(\log n) = c \log n(\log\log n - \log 2) + d \log n = c \log n \log\log n + (d - c\log 2)\log n \le c \log n \log\log n = O(c \log n \log\log n)$
(2) $T_3(n) = \Omega(\log n \log\log n)$, which can be proved in the same way.
Therefore, $T_3(n) = \Theta(\log n \log\log n)$.

(d) (4') $T_4(n) = T_4(\alpha n) + T_4(\beta n) + \Theta(n), (0 < \alpha + \beta < 1, \alpha, \beta > 0)$
Hint:

1. Think out binomial theorem. You may try to expand the expression using the recurrence relation several times. Find out the similarities between them.
2. Cases in (b) are different from those in (d) since the geometric series $\sum_{i=0}^{\infty}(\alpha+\beta)^i$ converges only when $|\alpha + \beta| < 1$.

Solution: Suppose $\alpha \le \beta$.

$$
\begin{aligned}
&T_4(n) = \textstyle\sum_{i=0}^{1} T\left(\binom{1}{i}\alpha^i\beta^{1-i}n\right) + \Theta(n) \\
&= \textstyle\sum_{i=0}^{2} T\left(\binom{2}{i}\alpha^i\beta^{2-i}n\right) + c\left(\sum_{i=0}^{1}\binom{1}{i}\alpha^i\beta^{1-i}n + n\right) \\
&= \textstyle\sum_{i=0}^{3} T\left(\binom{3}{i}\alpha^i\beta^{3-i}n\right) + c\left(\sum_{k=1}^{2}\sum_{i=0}^{k}\binom{k}{i}\alpha^i\beta^{k-i}n + n\right) \\
&= \textstyle\sum_{i=0}^{3} T\left(\binom{3}{i}\alpha^i\beta^{3-i}n\right) + cn\left(\sum_{k=1}^{2}\sum_{i=0}^{k}\binom{k}{i}\alpha^i\beta^{k-i} + 1\right) \\
&= \textstyle\sum_{i=0}^{3} T\left(\binom{3}{i}\alpha^i\beta^{3-i}n\right) + cn\left(\sum_{k=0}^{2}(\alpha+\beta)^k\right) \\
&= \cdots = cn\left(\sum_{k=0}^{\left\lceil \log_{\frac{1}{\beta}} n\right\rceil}(\alpha+\beta)^k\right) \\
&= cn\left(\frac{1-(\alpha+\beta)^{\left\lceil \log_{\frac{1}{\beta}} n\right\rceil+1}}{1-\alpha-\beta}\right) = \Theta(n).
\end{aligned}
$$